

Inbound-Call CRM Integration

Vendor Response

Prepared in response to: CRM Integration Requirements — Inbound Call / Lead Capture APIs (2026-06-16)

Our role in this integration: the CRM (system of record). Your Integration Platform originates every call; our platform receives, authenticates, stores, and reacts — agent screen-pop + billing.

Date: June 2026

1. Executive summary

Your specification describes a Salesforce-style inbound-call integration: an OAuth 2.0 Client-Credentials handshake followed by three REST touchpoints per call (Agent-of-Record lookup → lead post + agent screen-pop → billing update). The token-response fields you specified (`access_token`, `instance_url`, `id`, `token_type`, `scope`) and the Client-Credentials grant are the exact contract Salesforce exposes — so this is a pattern your platform already speaks, not a bespoke one-off.

Bottom line: we can support all eight of your Section 11 acceptance criteria — every one is a “Yes.” It is a net-new build, but it sits on capabilities our platform already runs in production: a data model with a built-in Agent-of-Record concept, a sub-second real-time push channel, an established secret-authenticated machine-to-machine API pattern, and encryption/secrets infrastructure. No existing authentication, commission, or policy logic is touched — the change is additive and low-risk.

This document gives you (a) a point-by-point response to your Section 11 criteria, (b) an honest map of what already exists versus what we build, (c) how the end-to-end flow works including the live agent screen-pop, (d) our security and scalability approach, and (e) the short list of questions we’d want to settle at onboarding.

2. Vendor response — Section 11 acceptance criteria

#	Requirement	Supported?	Notes
1	OAuth 2.0 Client-Credentials authentication with 24-hour cached bearer tokens	Yes	We stand up a token endpoint issuing a signed 24-hour bearer. Response shaped exactly to your spec (<code>access_token</code> , <code>instance_url</code> , <code>id</code> , <code>token_type</code> , <code>scope</code> , plus <code>expires_in</code>).
2	Token-authenticated GET lookup of Agent of Record by ANI (200 with agent ID / 204 if none)	Yes	We already model an Agent of Record on every client record. We add an indexed phone lookup so this responds well under your routing timeout. We return the caller’s Agent of Record for continuity; whether to route the call to them is your platform’s call (it knows the agent’s NetTrio availability).
3	Token-authenticated POST that finds/creates a lead and triggers an agent screen-pop	Yes	Find-or-create is idempotent (keyed on your requestTag). The screen-pop reuses the same real-time channel that already powers our in-app notifications.

#	Requirement	Supported?	Notes
4	Token-authenticated PATCH that updates the billable status on the existing lead record	Yes	Locates the record from the original POST and updates only billable. Tolerant of out-of-order delivery.
5	Accepts the shared lead data structure (Section 7) as JSON	Yes	We validate the structured fields (ani, state, billable, duration) and store classification fields (recordType, callProgram, subld) as provided.
6	Returns standard HTTP status codes (200 / 204 / 401) as described	Yes	In particular we return 401 (not 403) on an invalid/expired token, so your refresh-and-retry-once logic fires as designed.
7	HTTPS (TLS 1.2+) for all traffic	Yes	Our platform terminates TLS 1.2/1.3 by default on every endpoint; there is no plaintext path.
8	Prompt response time on the pre-call GET lookup to avoid routing timeouts	Yes	Engineered as the fast path: stateless token verification plus a single indexed query. We can point your platform directly at our endpoint to shave a hop if needed.

Honest framing: every item is “Yes,” but each is a net-new build on existing primitives — not something already running and exposed to you today. The engineering is well-understood and low-risk because the foundations are in place.

3. What we already have vs. what we build

A large share of this integration is reuse, which is why we’re confident on timeline and risk.

Already in production (reused)

- **A client/lead data model with an Agent of Record.** Every client record already carries its owning agent — exactly what your pcd maps to. Policies already link to clients.
- **A real-time push channel.** Our notification system pushes events to a specific signed-in user in roughly a tenth of a second over a live connection. The agent screen-pop is this same mechanism with a dialog instead of a bell badge.
- **An established machine-to-machine API pattern.** We already run inbound endpoints that authenticate external callers with a shared secret and constant-time comparison, then scope every database operation to Epic Life’s data explicitly.
- **Encryption, secrets, and rate-limiting infrastructure** used across the platform today; every request is automatically scoped to Epic Life’s data at the database level.

Net-new (built for this integration)

- **An OAuth token issuer** (we currently consume OAuth elsewhere; here we issue it) plus an API credential store.
- **The three data endpoints** (lookup / lead-post / billing) and a small call-event table recording each inbound call and its billing status.
- **A fast, indexed phone-number lookup** (client numbers are stored today but not optimized for exact-match lookup at call speed), and a stable pcd $\rightleftharpoons$ agent registry so your identifiers round-trip safely and always resolve to the right agent.

- **A new Clients page** to view and work these lead records, and the screen-pop dialog itself.

What we are not changing

No existing authentication, commission, or policy logic is modified. This is purely additive, which keeps the blast radius — and the risk to the rest of the system — low.

4. How it works, end to end

The flow mirrors your Section 8 process exactly: a once-daily authentication step, then three per-call touchpoints.

Integration Platform		Our CRM	
-----		-----	
(once / 24h)	POST /oauth/token	>	validate client_id + secret
		<.....	24h bearer token
PRE-CALL	GET /api/v1/leads?ani=	...>	indexed phone lookup (Epic Life)
		<.....	200 {pcId} or 204 (caller not on file)
ON ANSWER	POST /api/v1/leads	>	find-or-create lead; resolve agent
		<.....	by pcId --> SCREEN-POP to that agent
			200 OK (live, in-browser)
ON CALL END	PATCH /api/v1/leads	>	locate by requestTag, set billable
		<.....	200 OK (pop reflects call end)

1. **Authentication (once per 24h).** Your platform exchanges client_id + client_secret for a bearer token. You cache it for 24 hours per your spec, and refresh-and-retry once if you ever get a 401.
2. **Pre-call lookup.** We look the caller's number up against Epic Life's clients and return the Agent of Record's identifier (pcId), or 204 if the caller isn't on file. We don't decide availability — your call system (NetTrio) does; an agent who's turned calls off simply isn't routed one.
3. **Lead post + screen-pop.** We find-or-create the client (idempotently, keyed on your requestTag), write a call event, resolve which agent the call is routed to, and fire a live screen-pop into that agent's browser.
4. **Billing update.** At call end, your PATCH locates the same call event and records the final billable value. The open screen-pop reflects the call ending.

5. The two product surfaces you asked for

a) The Clients page

A new in-app page to view and work the lead/client records — searchable and sortable, showing each client's Agent of Record, status, linked policies, and a history of inbound calls (who called, when, billable status). It reuses our existing client data services and role-aware visibility (an agent sees their own book; an admin sees the whole team's).

b) The real-time agent screen-pop

When your platform routes a call to an agent, a dialog pops on that agent's screen within a fraction of a second, showing the caller and what we know about them, so the agent is oriented before they pick up. It updates live as the call ends.

We don't decide who's available or route calls — your call system (NetTrio) does. An agent turns inbound calls on or off in NetTrio; when they're off, your platform never routes them a call, so

we never pop for them. We simply react to the call your platform sends and show the screen-pop to the agent you routed it to.

One honest note: the screen-pop appears in our app, in the agent's browser, alongside NetTrio — the agent works with both open (NetTrio for the call, our app for the caller's history and context). If our app isn't open, the agent still takes the call in NetTrio; they just don't get the visual pop.

6. Security & data isolation

Your Section 10 requirements (TLS 1.2+, secrets stored/transmitted securely, never logged in clear text) are met, and we go further because a machine-to-machine integration has a specific risk profile:

- **TLS 1.2+ everywhere**, no plaintext path.
- **Client secrets are stored one-way hashed** (not reversibly encrypted) and shown to you once at creation, compared with a constant-time check. A database snapshot alone never yields a usable secret.
- **Bearer tokens are signed, time-boxed, and explicitly expiry-checked** with a dedicated signing key. An invalid or expired token gets a clean 401.
- **Every request is scoped to your data in code.** Because these calls carry no signed-in user, we don't rely on the usual database guards — each request is authenticated by its own token, and every read, write, and screen-pop is scoped to Epic Life's data and validated to resolve to a real Epic Life agent before anything happens.
- **Secrets and PII are never logged** — Authorization headers, tokens, and secrets are redacted; caller numbers are masked.
- **Per-credential rate limiting** guards against abuse and number-enumeration.

7. Scalability (built for thousands of users)

- **The pre-call lookup is the fast path** and is isolated as such: stateless token verification (no database hit just to authenticate) plus a single indexed phone lookup, on its own dedicated function so heavier traffic can never slow it down. Internal target: well under your routing timeout.
- **Idempotent, atomic writes** — your retry-once behavior and rapid repeat calls from the same number never create duplicates.
- **The real-time channel reuses our proven notification transport** for launch, with a clear path to a higher-throughput broadcast mechanism once concurrent call volume warrants it (a number we'd size with you).
- **Call records have a retention/archival plan from day one**, reusing the same approach we already run for call-recording storage.
- **On “do you have too many backend functions?” — no.** Each function is an independently deployed, scale-to-zero unit; the count of functions does not affect scalability, only actual traffic does.

8. Questions to confirm at onboarding

A few decisions are genuinely contract-level, and we'd rather settle them with you than assume:

1. **The subld field.** Your payload includes subld (sub-account / sub-publisher). Confirm what you'd like it to represent on our side — most likely a lead-source tag for reporting.
2. **Availability & routing stay with NetTrio.** We're assuming your platform (with NetTrio) decides who's available and routes the call, and we just receive it and pop the screen. Confirm that's right — and tell us if you'd ever want our app to reflect an agent's NetTrio on/off state (by default we don't need to).
3. **Your routing-timeout budget and expected concurrent-call volume** — these set our latency target and tell us when to move the screen-pop to the higher-throughput transport.
4. **Billing source of truth.** We treat the closing PATCH as authoritative for billable (the POST value is provisional). Please confirm that matches your model.

9. Anticipated questions (and our answers)

Q. What exactly is pclid, and who controls its format?

A. It's our identifier for the agent (the Agent of Record). We mint it and return it on the lookup; your platform echoes the same value back on the POST/PATCH so we know which agent to pop. We'll format it to match your existing convention (e.g. agent-00123).

Q. Can your pre-call lookup beat our routing timeout?

A. Yes, by design. We verify the token without a database round-trip and resolve the caller against a single indexed column. If we ever see margin risk, we point your platform straight at our endpoint to remove a network hop — your spec allows finalizing the base URL at onboarding.

Q. With no login on these machine calls, how is the data kept safe and correct?

A. Each request authenticates with its own token and is scoped to Epic Life's data; every call is validated to resolve to a real Epic Life agent before we record it or pop a screen. This is the same machine-to-machine discipline our existing inbound endpoints already use.

Q. What if your billing PATCH arrives but you never saw our POST (or they arrive out of order)?

A. We record the billing update anyway (an upsert keyed on requestTag), so a billable status is never silently dropped. Such a record is flagged so it doesn't fire a phantom screen-pop for a call that never posted.

Q. Where does the agent actually see the call?

A. A live screen-pop in our app, alongside NetTrio. Your platform routes the call (only to agents who have NetTrio on) and we pop the caller's details for that agent. If their app isn't open they still take the call in NetTrio — they just miss the visual pop.

Q. How big is this build and how risky is it?

A. Net-new but on proven foundations: a token endpoint, the three data endpoints, a small call-events table and credential store, an indexed phone lookup, a Clients page, and the screen-pop. No existing authentication, commission, or policy logic changes, so risk to the rest of the platform is low. The only genuine design choices are credential scope and no-Agent-of-Record routing.

Q. Are the client secret and tokens handled to your Section 10 standard?

A. Yes — secrets are hashed at rest with constant-time comparison, tokens are signed and expiry-checked, all traffic is TLS 1.2+, and secrets/tokens/PII are redacted from every log, including the error logging your spec calls for.

10. Edge cases we've already accounted for

Scenario	How we handle it
Billing PATCH arrives before the POST (out-of-order)	Upsert on requestTag — billing is recorded, never dropped; no phantom pop.
Duplicate POST (your retry-once, or a network retry)	Idempotent on requestTag — one lead, one call record, one pop.

Scenario	How we handle it
Agent has inbound calls turned off in NetTrio	Your platform doesn't route them a call, so we never pop for them. We don't track or override NetTrio status — it's your source of truth.
Caller number formatting (leading "1", dashes, spaces)	Normalized identically on both sides so the lookup matches.
Brand-new caller (no client record yet)	Lookup returns 204; the POST creates the client and routes the call.
Unfamiliar recordType value	Stored as provided — onboarding workflow changes won't break us.
Token expires mid-call (POST near hour 24, PATCH after)	We return 401; your refresh-and-retry-once succeeds.
Call-record PII and retention	Defined retention window; numbers masked and secrets/tokens redacted in logs.

11. Summary

This integration is a strong fit for our platform. We already run the hard parts — an Agent-of-Record data model, a sub-second real-time push channel, a secure machine-to-machine API pattern, and the encryption/secrets infrastructure to back it. The work is to expose them through the Salesforce-style contract your platform already uses, add a fast phone lookup, and build the Clients page and screen-pop on top. Every Section 11 criterion is a “Yes,” the change is additive and low-risk, and the only open items are a couple of details to confirm at onboarding (the subId mapping and billing authority).

Sample values throughout your specification (domains, IDs, secrets) are understood to be illustrative; production values will be issued during onboarding.